



Tập 2: CNI Specification

Lý thuyết: CNI Operations, cấu trúc `.conflist` và IPAM

Network Thực Chiến · Series: Kubernetes Networking · Tập 02

CNI là gì? Tại sao tồn tại?

CNI = Container Network Interface — một **đặc tả kỹ thuật** (specification) mở, không phải sản phẩm cụ thể.

Bài toán: Kubernetes cần giao tiếp với **nhiều hệ thống mạng khác nhau** (Flannel, Calico, Cilium, Weave...) mà không muốn viết code riêng cho từng hệ thống.

```
kubelet — (gọi theo chuẩn CNI) —> CNI Plugin  
          "Làm ơn cấp IP cho Pod này!"   (Flannel / Calico / Cilium...)
```

CNI định nghĩa **giao diện giao tiếp** giữa Container Runtime (kubelet) và Network Plugin.

CNI: Mô hình Stateless Binary — Không phải Daemon

CNI plugin = **binary file thuần** trong `/opt/cni/bin/`. Không có process nào chạy nền.

Kubelet cần ADD:

```
fork + exec
/opt/cni/bin/bridge
↓ env vars
↓ config (stdin)
↓ plugin chạy
↓ trả JSON (stdout)
plugin process exit ✓
```

Kubelet cần DEL:

```
fork + exec
/opt/cni/bin/bridge ← process MỚI hoàn toàn
↓ env vars
↓ config (stdin)
↓ plugin chạy
↓ trả kết quả
plugin process exit ✓
```

So sánh với CRI (containerd):

	CRI (containerd)	CNI plugin
Mô hình	gRPC daemon, Unix socket	Stateless binary, exec mỗi lần
Process	1 process chạy mãi	1 process/operation, exit sau khi xong
State	Trong memory	Phải ghi ra file (<code>/var/lib/cni/networks/</code>)

Hệ quả: Vì plugin không có memory state, nếu DEL không chạy (Node crash), IP bị giữ mãi trong file. Đây là lý do CNI v1.1.0 thêm operation **GC** — dọn state file không còn chủ.

CNI v1.1.0: Operations (không phải "Verbs")

Thuật ngữ chính thức trong CNI spec là **"operations"**, không phải "verbs".

Mỗi operation được truyền vào plugin qua biến môi trường `CNI_COMMAND`.

CNI spec v1.1.0 định nghĩa **6 operations** chia thành 3 nhóm:

Lifecycle — quản lý vòng đời Pod:

Operation	CNI_COMMAND	Mục đích
ADD	<code>ADD</code>	Cấp IP, tạo veth pair, cấu hình routing khi tạo Pod
DEL	<code>DEL</code>	Giải phóng IP, dọn dẹp veth, xóa routes khi xóa Pod
CHECK	<code>CHECK</code>	Xác minh cấu hình mạng Pod còn đúng (kubelet sync)

Maintenance — mới trong v1.1.0 (giải quyết resource leak khi Node crash):

Operation	CNI_COMMAND	Mục đích
GC	<code>GC</code>	Dọn dẹp tài nguyên mạng của Pod đã mất
STATUS	<code>STATUS</code>	Kiểm tra CNI plugin có sẵn sàng nhận lệnh không

CNI v1.1.0: Operations (2)

Meta — introspection:

Operation	CNI_COMMAND	Mục đích
VERSION	VERSION	Query xem plugin hỗ trợ CNI spec version nào

Lưu ý: Chính spec v1.1.0 cũng ghi nhầm "5 operations" trong phần overview nhưng thực tế document đủ 6.

VERSION thường bị bỏ qua vì không liên quan trực tiếp đến lifecycle của Pod.

Luồng ADD khi tạo Pod

```
kubectl apply -f pod.yaml
```

↓
[1] API Server lưu Pod spec vào etcd

↓
[2] Scheduler chọn Node → Ghi nodeSelector vào etcd

↓
[3] kubelet trên Node đó nhận event

↓
[4] kubelet tạo pause container (giữ network namespace)

↓
[5] kubelet đọc /etc/cni/net.d/*.conflist
→ Xác định CNI plugin nào cần gọi

↓
[6] kubelet EXEC binary CNI plugin với CNI_COMMAND=ADD
Truyền vào: network namespace path, Pod name, container ID

↓
[7] Plugin cấp IP, tạo veth, cấu hình iptables/eBPF
Trả về: JSON chứa IP, routes, DNS info

↓
[8] Pod sẵn sàng với IP! ✅

CRI ↔ CNI: Ai tạo Network Namespace?

Nhầm lẫn phổ biến: **CNI plugin tạo network namespace** — SAI. Thứ tự thực tế:

```
kubectl apply -f pod.yaml
```

↓

[kubelet] —gRPC→ [containerd (CRI daemon)]

- 1. Tạo pause container
- 2. Tạo Network Namespace ← CRI làm, không phải CNI!

Path: /proc/<PID>/fd/4

↓

[kubelet] ← "Xong. netns = /proc/123/fd/4"

↓

[kubelet] —exec→ [/opt/cni/bin/bridge] ← stateless binary

CNI_NETNS=/proc/123/fd/4 ← nhận path từ kubelet

CNI_COMMAND=ADD

↓

Vào netns, tạo eth0
Cấp IP qua IPAM
Thêm routes

Tóm tắt trách nhiệm:

- **CRI (containerd)**: tạo và sở hữu Network Namespace, tạo pause container
- **CNI plugin**: nhận netns path đã có sẵn, chỉ cấu hình interface/IP/routes bên trong
- **kubelet**: orchestrator — gọi CRI trước, sau đó gọi CNI với path từ CRI

Cấu trúc File `.conflist` — Chained Plugins

`/etc/cni/net.d/10-flannel.conflist` (ví dụ thực tế):

```
{
  "cniVersion": "1.0.0",
  "name": "cbr0",
  "plugins": [
    {
      "type": "flannel",           ← Plugin chính: cấp IP, tạo veth
      "delegate": {
        "hairpinMode": true,
        "isDefaultGateway": true
      }
    },
    {
      "type": "portmap",         ← Plugin 2: xử lý hostPort mapping
      "capabilities": {
        "portMappings": true
      }
    },
    {
      "type": "bandwidth",       ← Plugin 3: giới hạn bandwidth (optional)
      "capabilities": {
        "bandwidth": true
      }
    }
  ]
}
```

Chained Plugins: Tại sao lại chuỗi?

CNI cho phép **xếp chồng nhiều plugin** — kết quả của plugin trước là đầu vào của plugin sau:

```
kubelet gọi ADD
|
▼
[Plugin 1: bridge]
  ✓ Tạo bridge mylab0
  ✓ Tạo veth pair
  ✓ Cấp IP qua host-local IPAM
  ✓ Thêm routes
    | Kết quả (interface, IP) truyền xuống
    ▼
  [Plugin 2: portmap]
    ✓ Tạo iptables rule cho hostPort mapping
    |
    ▼
  [Plugin 3: firewall]
    ✓ Tạo iptables rule chặn spoofing IP
    |
    ▼
  Trả về JSON kết quả tổng hợp cho kubelet
```

DEL Flow & Error Recovery trong Chained Plugins

DEL khi Pod bị xóa — thứ tự NGƯỢC với ADD:

```
Pod bị xóa
  |
  ▼
[Plugin 3: firewall] DEL → xóa iptables anti-spoof rules
  |
  ▼
[Plugin 2: portmap] DEL → xóa hostPort iptables rules
  |
  ▼
[Plugin 1: bridge] DEL → xóa veth pair, trả IP về IPAM pool
  |
  ▼
kubelet báo containerd: destroy pause container + network namespace
```

Nếu ADD thất bại giữa chừng — kubelet tự động rollback:

```
[Plugin 1: bridge] ADD ✓ tạo veth, cấp IP 10.99.0.2
[Plugin 2: portmap] ADD ✗ FAIL! (config sai, iptables lỗi...)
```

```
Kubelet detect failure → gọi DEL theo thứ tự ngược:
  [Plugin 2: portmap] DEL (no-op, chưa cấu hình gì)
  [Plugin 1: bridge] DEL ✓ giải phóng 10.99.0.2, xóa veth
```

→ Không có partial state leak!

IPAM: Quản lý địa chỉ IP

IPAM (IP Address Management) là sub-plugin chịu trách nhiệm **cấp và thu hồi IP**:

IPAM Plugin	Cơ chế lưu trữ IP đã cấp	Phù hợp với
host-local	File trong <code>/var/lib/cni/networks/</code>	Dev/Lab (đơn giản, per-node)
dhcp	DHCP server truyền thống	Môi trường tích hợp datacenter
calico-ipam	etcd của Calico cluster	Production với Calico CNI
whereabouts	CRD trong K8s	Multus, multi-network

Operation GC: Giải pháp cho Resource Leak

Vấn đề trước CNI v1.1.0: Khi Node bị crash đột ngột:

- Pod bị terminate nhưng CNI không kịp gọi **DEL**
- Kết quả: IP bị giữ mãi trong `/var/lib/cni/networks/`, veth "zombie" còn tồn tại trên Node

Giải pháp — Operation GC (v1.1.0):

```
# kubelet định kỳ gọi GC với danh sách container ID còn sống
# Plugin so sánh với state file, xóa những thứ không còn cần
cnitool gc mylab-network \
  --valid-attachments='[{"containerID":"abc123"}, {"containerID":"def456"}]'
```

GC giống như "garbage collector" — dọn sạch tài nguyên mạng của Pod đã chết mà DEL chưa kịp chạy.

Operation STATUS: Health Check

STATUS (mới từ v1.1.0) cho phép kubelet kiểm tra xem CNI plugin có **sẵn sàng** nhận lệnh không:

```
# Kubelet gọi STATUS trước khi schedule Pod lên Node
cnitool status mylab-network
# Exit code 0 = Plugin OK, sẵn sàng
# Exit code 1 = Plugin đang bị lỗi, KHÔNG schedule Pod lên Node này!
```

Trước đây, nếu CNI plugin bị lỗi (daemon crash, config sai), kubelet vẫn cố tạo Pod và thất bại ở bước ADD — mất thời gian và gây confusion. STATUS giải quyết điều này bằng cách **phát hiện sớm**.

Biến môi trường khi kubelet gọi CNI

Kubelet truyền thông tin cho CNI plugin qua **biến môi trường**:

```
CNI_COMMAND=ADD           # Verb: ADD | DEL | CHECK | GC | STATUS
CNI_CONTAINERID=abc123     # ID của container (pause container)
CNI_NETNS=/proc/123/fd/4   # Path đến network namespace của container
CNI_IFNAME=eth0            # Tên interface sẽ tạo bên trong Pod
CNI_PATH=/opt/cni/bin      # Thư mục chứa CNI binary plugins
CNI_ARGS=K8S_POD_NAME=my-pod;K8S_POD_NAMESPACE=default
```

Plugin nhận config mạng qua **stdin** (JSON format từ file .conflist).

Tổng kết Tập 2

Khái niệm	Tóm tắt
CNI	Đặc tả giao diện giữa kubelet và network plugin
Stateless binary	Exec mới mỗi operation, không phải daemon — state phải lưu ra file
CRI tạo netns	containerd tạo network namespace; CNI nhận path có sẵn, chỉ cấu hình bên trong
Operations	Thuật ngữ chính thức — truyền qua <code>CNI_COMMAND</code> env var
ADD/DEL/CHECK	3 lifecycle operations: cấp IP, thu hồi, kiểm tra
DEL idempotent	Plugin phải xử lý được khi netns đã không còn — không return error
GC/STATUS	2 maintenance operations mới v1.1.0 (chống resource leak)
VERSION	Meta operation: query spec version plugin hỗ trợ
Chained plugins	Nhiều plugin xếp chồng — ADD fail → kubelet rollback DEL ngược chiều
IPAM	Sub-plugin quản lý việc cấp/thu hồi IP

Bài Lab 1.2: Tự tay viết `.conflist` và gọi ADD/DEL bằng `cnitool` trực tiếp!

👉 Chuyển sang Lab 1.2

Mở file `lab-guide.md` trong thư mục `1.2/` để thực hành:

- Cài CNI plugin binaries và `cnitool`
- Viết file `.conflist` với chained plugins (bridge + portmap)
- Gọi `cnitool add` thủ công và quan sát kết quả
- Gọi `cnitool del` và kiểm tra resource được giải phóng